

Relatório de Auditoria de Segurança

daily-divine-word

01/09/2026

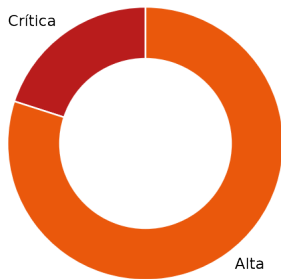
Escopo: frontend React/Vite/TypeScript, backend Nitro, acesso PostgREST do Supabase, migrations, configuração Vercel, histórico Git e bundle de produção.

Nota metodológica: as cinco categorias foram mapeadas para a stack detectada. RLS foi tratado como mecanismo de isolamento do banco; como não há autenticação de usuário/tenant, as rotas Nitro foram auditadas como fronteira de autorização. Não foram considerados achados hipotéticos.

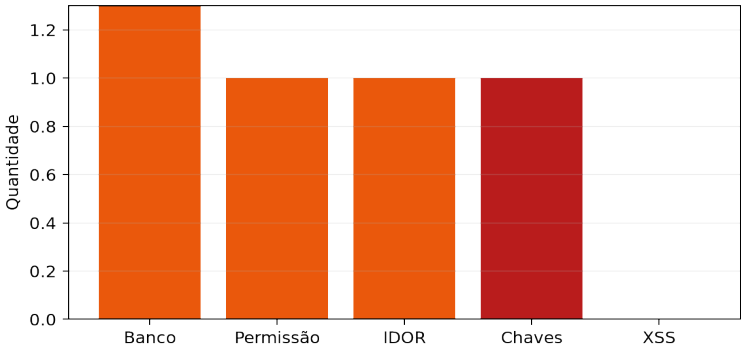
Resumo executivo

Foram verificados 4 achados de segurança: 1 crítico e 3 altos. A categoria XSS não apresentou ocorrência verificável no código auditado.

Achados por severidade



Achados por categoria



Stack detectada

React 18 + Vite + TypeScript; Nitro server routes; Supabase via REST/PostgREST com fetch; sem ORM/query builder; sem autenticação implementada; RLS nas tabelas; Vercel via vercel.json; sem Docker, Helm, Terraform ou CI configurados.

Pontos fortes

- RLS está habilitado em whatsapp_cadastros e whatsapp_servicos, com políticas deny para anon/authenticated e sem chave service_role no bundle frontend.
- A validação Zod é estrita, há limite de corpo, rate limit por IP, normalização de telefone e filtro atômico status=ATIVO para evitar recancelamento.
- Não foram encontrados innerHTML, dangerouslySetInnerHTML, v-html, eval, new Function, markdown/HTML sem sanitização ou URLs javascript: no frontend/backend auditados.
- .env.example não contém valor secreto e .gitignore inclui .env; o problema é o arquivo histórico já rastreado.

Pontos fracos centrais

A ausência de autenticação de atendente torna a API de cancelamento pública e faz com que dados enviados pelo navegador sejam tratados como identidade e autorização. A service_role concentra privilégio no servidor, mas não existe uma fronteira de usuário/tenant para limitar a operação. A exposição histórica da chave exige rotação antes de qualquer publicação.

Achados detalhados

Severidade	Arquivo:linha	Descrição
Alta	Supabase remoto: public.rls_auto_enable() (linha não aplicável)	Banco / função SECURITY DEFINER O advisor de segurança do projeto detecta uma função SECURITY DEFINER no schema public executável por anon e authenticated via RPC. Não há arquivo local correspondente para revisar sua implementação. Trecho: Função public.rls_auto_enable(), SECURITY DEFINER, EXECUTE público (advisor Supabase) Impacto: A função pode operar com privilégios do criador e está exposta a papéis não autenticados; o impacto exato depende do corpo da função.
Alta	server/routes/api/cancelamento.ts:13-40	Banco sem tranca / isolamento A rota de cancelamento é pública (não há autenticação, tenant ou dono) e altera serviços usando credenciais service_role no servidor. O RLS do Supabase bloqueia anon/authenticated, mas é bypassado pela service_role. Trecho: if (event.method !== "POST") ...; await cancelServico(parsed.data.serviceld, {...}) Impacto: Qualquer pessoa que conheça ou tente IDs pode cancelar dados de outros clientes e forjar os metadados de auditoria.
Alta	server/routes/api/cancelamento.ts:38; src/components/liturgya/WhatsAppRegistration.tsx:364-369	Permissão definida no navegador O navegador coleta responsibleUser, reason e confirm, mas o servidor não valida identidade, papel ou vínculo do usuário responsável; aceita qualquer texto enviado por um cliente não autenticado. Trecho: cancelado_por: parsed.data.responsibleUser; motivo_cancelamento: parsed.data.reason Impacto: Um atacante pode executar a operação privilegiada e atribuí-la a qualquer atendente, sem trilha confiável.
Alta	server/utis/supabase.ts:50-51; server/routes/api/cancelamento.ts:31-38	IDOR serviceld vem do body e é interpolado diretamente na query PATCH. Não existe verificação server-side de que o serviço pertence ao telefone consultado, ao cliente ou ao tenant do chamador. Trecho: whatsapp_servicos?id=eq.\${encodeURIComponent(serviceld)}&status=eq.ATIVO Impacto: Com um UUID de outro serviço, é possível cancelar o registro de terceiro; o filtro status=ATIVO só evita recancelamento, não impede acesso indevido.
Crítica	.env:3; histórico Git; GitHub main/.env	Chaves expostas O arquivo .env está rastreado localmente e contém uma chave service_role JWT. O mesmo caminho está presente no histórico Git; o arquivo remoto do branch main respondeu HTTP 200. Trecho: NITRO_SUPABASE_SERVICE_ROLE_KEY= Impacto: A chave concede acesso privilegiado ao banco e deve ser considerada comprometida.

Recomendações priorizadas

P1	Revogar e rotacionar a service_role exposta; remover .env de todo o histórico Git antes de qualquer push/deploy.
P2	Implementar autenticação de atendentes e autorização server-side por organização/tenant; nunca aceitar responsibleUser como identidade.
P3	Trocar o PATCH por operação transacional/RPC que valide service_id + cliente + escopo autorizado, preservando concorrência.
P4	Adicionar testes de autorização negativa, IDOR, concorrência e verificação do bundle/histórico no CI.
P5	Configurar secrets na Vercel em Production/Preview e revisar logs para garantir que nenhum segredo seja impresso.

ISSUES PARA O GITHUB

As issues abaixo estão prontas para copiar e colar. O achado XSS não gera issue porque não foi verificado.

```
--- ISSUE 1 ---
Título: [Segurança] Banco / função SECURITY DEFINER
Labels sugeridas: security, alta

## Descrição
O advisor de segurança do projeto detecta uma função SECURITY DEFINER no schema public executável por anon e authenticated via RPC. Não há arquivo local correspondente para revisar sua implementação.

## Evidência
- Supabase remoto: public.rls_auto_enable() (linha não aplicável)
- Trecho: `Função public.rls_auto_enable(), SECURITY DEFINER, EXECUTE público (advisor Supabase)`

## Impacto
A função pode operar com privilégios do criador e está exposta a papéis não autenticados; o impacto exato depende do corpo da função.

## Sugestão de correção
Inspecionar o corpo, mover para schema não exposto ou trocar para SECURITY INVOKER quando possível e revogar EXECUTE de anon/authenticated.

## Critérios de aceite
- [ ] A operação exige autenticação e autorização server-side.
- [ ] Teste negativo impede acesso fora do escopo.
- [ ] Logs não registram segredos.
- [ ] Testes automatizados e build passam.
--- FIM ISSUE 1 ---

--- ISSUE 2 ---
Título: [Segurança] Banco sem tranca / isolamento
Labels sugeridas: security, alta

## Descrição
A rota de cancelamento é pública (não há autenticação, tenant ou dono) e altera serviços usando credenciais service_role no servidor. O RLS do Supabase bloqueia anon/authenticated, mas é bypassado pela service_role.

## Evidência
- server/routes/api/cancelamento.ts:13-40
- Trecho: `if (event.method !== "POST") ...; await cancelServico(parsed.data.serviceId, {...})`

## Impacto
Qualquer pessoa que conheça ou tente IDs pode cancelar dados de outros clientes e forjar os metadados de auditoria.

## Sugestão de correção
Adicionar autenticação de atendente, autorização por organização e executar a operação em transação/RPC com contexto autorizado.

## Critérios de aceite
```

```

- [ ] A operação exige autenticação e autorização server-side.
- [ ] Teste negativo impede acesso fora do escopo.
- [ ] Logs não registram segredos.
- [ ] Testes automatizados e build passam.
--- FIM ISSUE 2 ---

--- ISSUE 3 ---
Título: [Segurança] Permissão definida no navegador
Labels sugeridas: security, alta

## Descrição
O navegador coleta responsibleUser, reason e confirm, mas o servidor não valida identidade, papel ou vínculo do usuário responsável; aceita qualquer texto enviado por um cliente não autenticado.

## Evidência
- server/routes/api/cancelamento.ts:38; src/components/liturgy/WhatsAppRegistration.tsx:364-369
- Trecho: `cancelado_por: parsed.data.responsibleUser; motivo_cancelamento: parsed.data.reason`

## Impacto
Um atacante pode executar a operação privilegiada e atribuí-la a qualquer atendente, sem trilha confiável.

## Sugestão de correção
Autenticar o atendente no backend e derivar o usuário a partir da sessão; rejeitar responsibleUser arbitrário.

## Critérios de aceite
- [ ] A operação exige autenticação e autorização server-side.
- [ ] Teste negativo impede acesso fora do escopo.
- [ ] Logs não registram segredos.
- [ ] Testes automatizados e build passam.
--- FIM ISSUE 3 ---

--- ISSUE 4 ---
Título: [Segurança] IDOR
Labels sugeridas: security, alta

## Descrição
serviceId vem do body e é interpolado diretamente na query PATCH. Não existe verificação server-side de que o serviço pertence ao telefone consultado, ao cliente ou ao tenant do chamador.

## Evidência
- server/utils/supabase.ts:50-51; server/routes/api/cancelamento.ts:31-38
- Trecho: `whatsapp_servicos?id=eq.${encodeURIComponent(serviceId)}&status=eq.ATIVO`

## Impacto
Com um UUID de outro serviço, é possível cancelar o registro de terceiro; o filtro status=ATIVO só evita recancelamento, não impede acesso indevido.

## Sugestão de correção
Consultar o serviço por id + cliente/tenant autorizado e aplicar a alteração em uma operação atômica com ownership.

## Critérios de aceite
- [ ] A operação exige autenticação e autorização server-side.
- [ ] Teste negativo impede acesso fora do escopo.
- [ ] Logs não registram segredos.
- [ ] Testes automatizados e build passam.
--- FIM ISSUE 4 ---

--- ISSUE 5 ---
Título: [Segurança] Chaves expostas
Labels sugeridas: security, crítica

## Descrição
O arquivo .env está rastreado localmente e contém uma chave service_role JWT. O mesmo caminho está presente no histórico Git; o arquivo remoto do branch main respondeu HTTP 200.

## Evidência
- .env:3; histórico Git; GitHub main/.env
- Trecho: `NITRO_SUPABASE_SERVICE_ROLE_KEY=`

## Impacto
A chave concede acesso privilegiado ao banco e deve ser considerada comprometida.

## Sugestão de correção

```

Revogar/rotacionar a chave, remover .env de todo o histórico, fazer force-with-lease controlado e usar somente secrets da Vercel/ambiente.

Critérios de aceite

- [] A operação exige autenticação e autorização server-side.
- [] Teste negativo impede acesso fora do escopo.
- [] Logs não registram segredos.
- [] Testes automatizados e build passam.

--- FIM ISSUE 5 ---